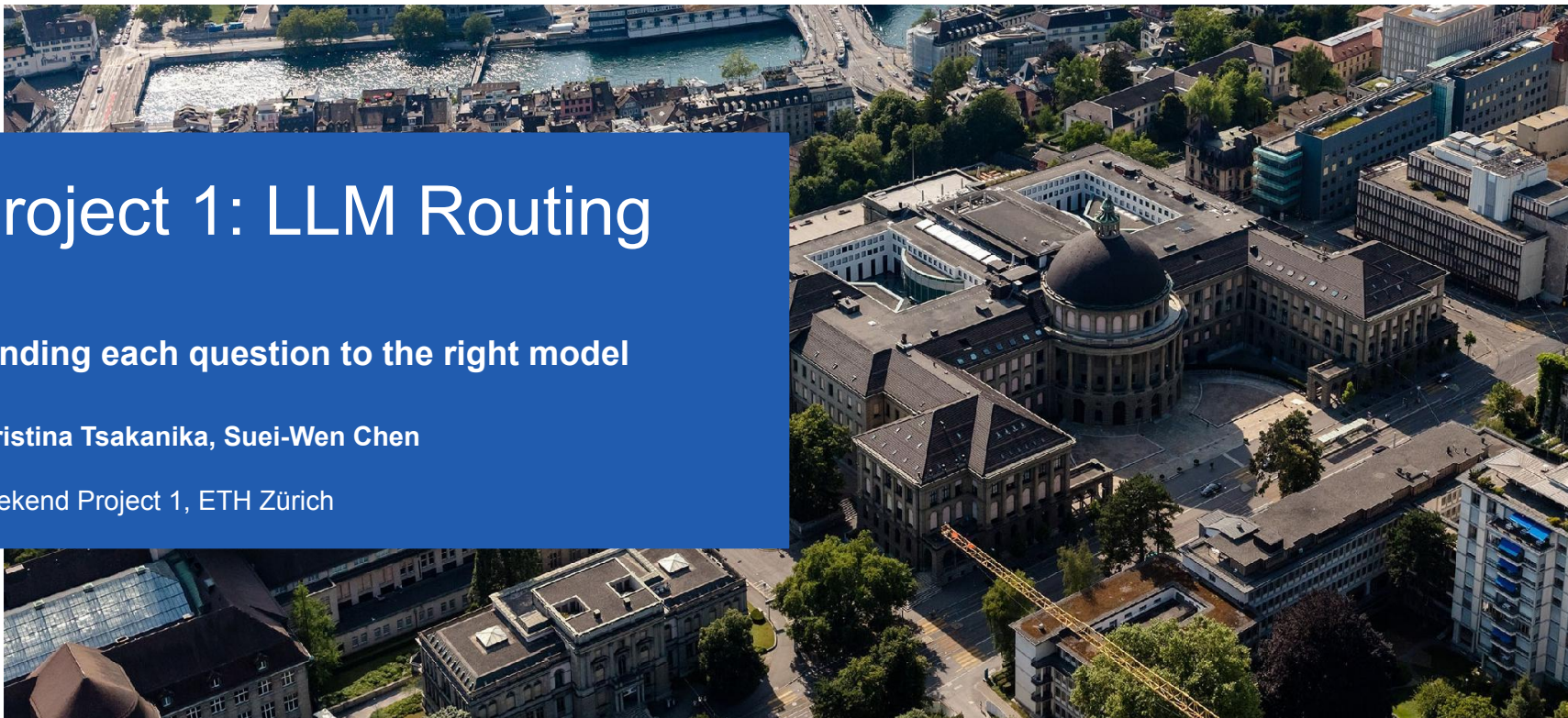


Project 1: LLM Routing

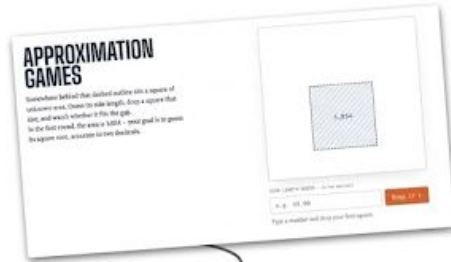
Sending each question to the right model

Christina Tsakanika, Sui-Wen Chen

Weekend Project 1, ETH Zürich



What you have seen this week



A PATH FROM APPROXIMATION GAMES TO NEURAL NETWORKS



Neural Networks as Universal Approximators: Rebuilding VeloZürli's Demand Map

The situation

You have just taken over as **Head of City Operations at VeloZürli**, the bike-sharing operator that runs 12,000 bikes across 380 docks in Zürich. Two decisions are sitting on your desk this week, and both of them are really the same decision in disguise.

First, the rebalancing vans. Every night a fleet of vans redistributes bikes so that each dock starts the morning with roughly what it will need. Right now the schedule is a spreadsheet somebody built in 2019 out of round numbers. Your operations lead, Nadia, wants to lada, wants to know: *how many bikes should the Hauptbahnhof dock hold at each hour of the day?*

Weekend project: LLM routing

A large language model is a neural network with billions of parameters, trained on huge amounts of text to predict the next word.

LLM routing

The process of directing a prompt, request or task to the most appropriate large language model, based on specific factors.

complexity

cost

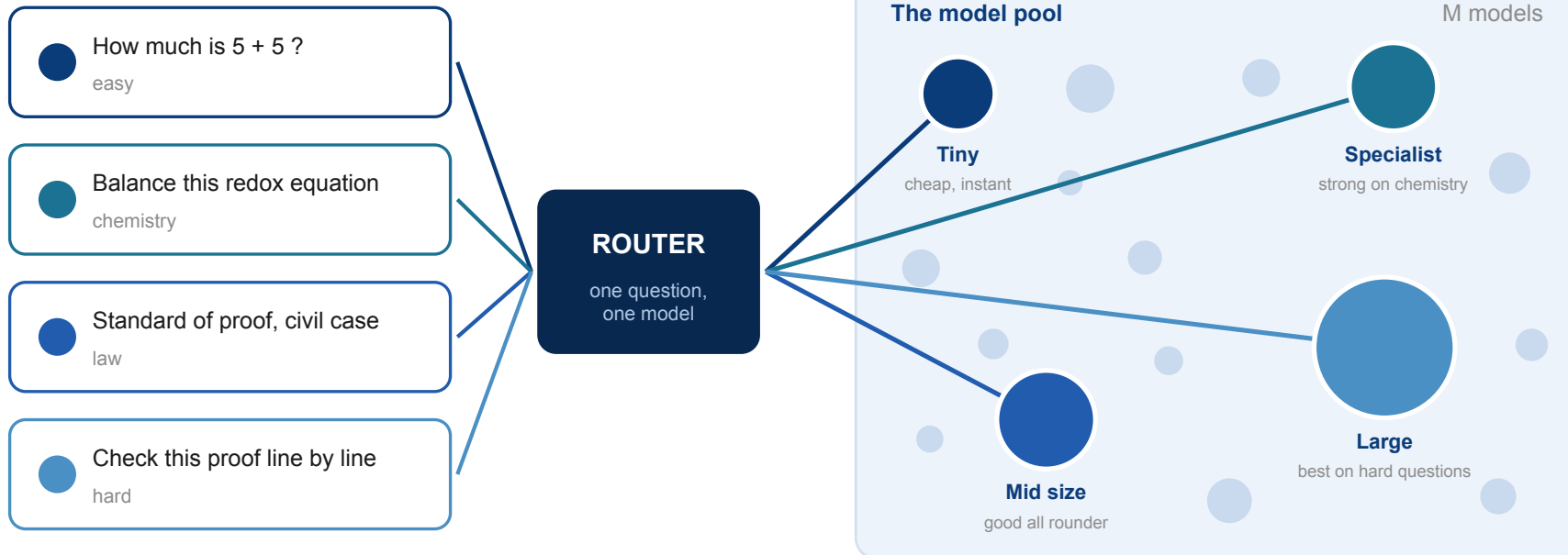
latency

data sensitivity

output quality

Routing, in one picture

Questions arriving



Benefits of Routing

1 Cost control

You stop paying frontier prices for questions a small model answers correctly.

2 Better answers

Some smaller models are specialists and beat the largest model on their own topic.

3 Lower latency

A small model replies in a fraction of the time, which matters for short questions.

4 Resilience

If one model or provider goes down, the routing logic sends the request to an approved fallback.

A real system: OpenRouter

Commercial gateway that puts hundreds of models behind a single API endpoint.
It handles fallbacks and can pick the model for you.

The request you send

```
{  
  "model": "openrouter/auto-beta",  
  "messages": [ { "role": "user",  
    "content": "your question" } ]  
}
```

The model is decided by the router
The response returns us the answer.

How it decides

- 1 Classify the task**
a small fast network sorts the prompt into one of about 30 task types, such as code debugging, maths, or customer support
- 2 Rank by real usage**
for that task type, which models developers actually keep using
- 3 Apply your cost tier**
the quality against cost dial you set
- 4 Route, with fallbacks**
if the pick is unavailable, go to the next one

Congratulations! You have been hired

RelayAI is an AI tutor. Engineering students at ETH use it, medical students at UZH use it, law students use it, and ... so do a few thousand people we did not expect!

Tens of thousands of questions arrive every day, and they are nothing alike.

Balance this redox equation.

What is the standard of proof in a civil case?

Check this proof, line by line.

Goal

Receiving these questions, return the model with the lowest cost that still manages to give a good answer.

You work on the router: the piece that takes a question and decides which model answers it.

How do we do that?

We train a neural network that takes the prompt as input and returns an error estimate for every model in the pool.



We call this predicted error γ . It is a number between 0 and 1: how likely that model is to get this question wrong.

Now we know the predicted error, and the cost of every model is already on the price list.

The routing rule

$$r(x) = \operatorname{argmin\,over\,} m \left[\gamma^m(x) + \lambda \cdot c^m \right]$$

- x** the incoming question
- m** one of the M models in the catalog
- c^m** the cost of calling model m, known and fixed
- λ** the client's dial, high means save capacity
- $\gamma^m(x)$** the predicted error of model m on question x
- $r(x)$** the single model we decide to call

c is on the price list (known).
 λ comes from the client (defined $\rightarrow$ known).
The one thing we have to build is γ .

Model Input: embeddings

A neural network cannot understand human language. It needs a numeric representation of the text. That representation is called an embedding.

"Balance this redox equation."



0.12 -0.44 0.07 0.31 -0.09 0.58 ... -0.21

a fixed list of 384 numbers

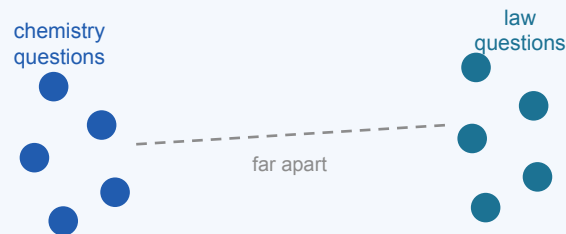
Meaning becomes distance

Questions with a similar semantic meaning sit close together in this mathematical space, even when they share no words at all. Questions about different things sit far apart.

The embeddings are handed to you (embeddings.npy).

In the code this is Z, with one row per question and 384 columns.

the embedding space



The data you are given

The dataset is labelled: for every question and every model we recorded whether the model got it right. That is what makes learning possible.

`prompts.csv.gz`

the questions, their topic, and their train, validation or test split

`models.csv`

the catalog: name, size, cost, and which pool each model belongs to

`errors.npy`

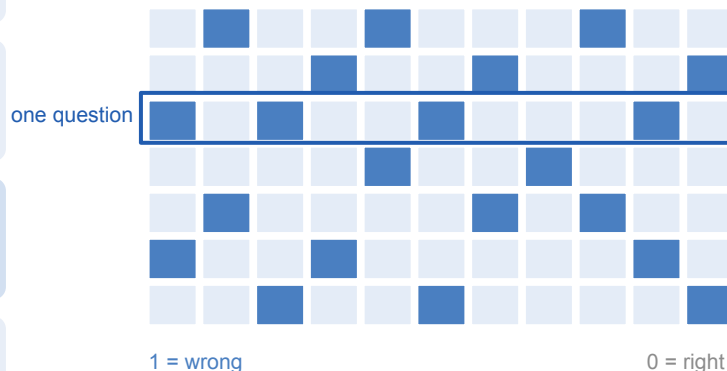
the “grade book” L: 0 if the model got it right, 1 if it got it wrong

`embeddings.npy`

$\phi(\mathbf{x})$ 384 numbers per question

The “grade book” L

rows are questions, columns are models



35,673

questions

112

models

11

topics

384

numbers per question

The network you train

$$\gamma_{\text{mlp}}(\mathbf{x}) = \sigma(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \boldsymbol{\varphi}(\mathbf{x}) + \mathbf{b}_1) + \mathbf{b}_2)$$



What it learns from

Loss = mean over all pairs of $(\gamma^m(\mathbf{x}) - \ell^m(\mathbf{x}))^2$

At test time

The router meets questions it has never seen. Nothing is learned any more, the weights are frozen.

1. An unseen question

not in the training set

2. $\phi(x)$

384 numbers

3. $y(x)$

one predicted error per model

4. $y + \lambda \cdot c$

one score per model

5. argmin

one model is called

The project part by part

Part 0

Get the data

Load the four files, meet the catalog, and learn the names used everywhere below.

Part 1

Look before you build

Guided data visualisations.

Part 2

The router

Build γ with the two layer network, train it, and evaluate it.

Demo time

**RelayAI Router Explorer**
Every question to the model that should answer it

lambda 0.20070 models in the poolMLP 384 → 128 → 7058,310 weights

AT LAMBDA 0.20, ON THE SEALED TEST SPLITthe router answers at **55.5%** accuracy for a mean cost of **0.627** | always calling SUS-Chat-34B: **55.6%** at **0.849** | it spreads the work over **39** of 70 models

PANEL 1
Ask
Pick an incoming question or type your own, then set the dial.

TODAY'S QUESTIONS click again for another one
One question per topic, drawn from the held out split. Hover one for the exact text that gets embedded: some carry a few-shot primer, and the trailing **A:** is the model's cue to answer, not a missing answer.

**Medicine and biology**
asked by a UZH medical student
The main reason given by men and women for their long lasting marriage is that

**Grade school math**
asked by a high school student
At the arcade Debby spent 2 tickets on a hat, 10 on a stuffed animal and 2 on...

**Advanced math**
asked by a maths student, late in term
a sum of money at simple interest amounts to rs . 835 in 3 years and to rs

TYPE YOUR OWN QUESTION
Ask anything, for example: how do I balance a redox equation in acidic solution?

PANEL 2
Decision
 $\text{gamma} = \text{sigmoid}(W2 \text{ReLU}(W1 \text{phi}(x) + b1) + b2)$, then plus lambda x cost, then argmin.

THE QUESTION reads like Truthfulness traps
The following are multiple choice questions (with answers) about human aging.
The main reason given by men and women for their long lasting marriage is that
A. They are best friends
B. They must fulfill their vows
C. They agree on almost everything

THE ROUTER CALLS
Starling-LM-7B-beta
7Bcost 0.53catalog accuracy 48.1%
predicted error 52% of the score, so predicted accuracy **88.6%** price 48%

HOW THE NUMBER WAS BUILT

1 hidden layer
 $\text{ReLU}(W1 \text{phi}(x) + b1)$, 384 → 128

57 of 128 units fired, the rest ReLU set to zero
shared: every model in the pool reads this same list

2 output layer
 $\text{sigmoid}(W2 \text{h} + b2)$, 128 → 70
models scored
lowest gamma **0.072**
winner gamma **0.114**
pool average **0.251**
one row of W2 per model, 129 numbers each

3 plus lambda x cost, argmin
lambda = 0.200
winning score **0.220**
margin over runner up **0.0023**
exactly one model is called

Practical things

1 Office hours

2 Deadline

Sunday 25 October 2026, 23:59

3 Where to ask

TAs!

Good luck!